

OE-EC604B — Operating System

Semester VI • 3L:0T:0P • 3 Credits

Module 9 Exam Notes: Protection, Security, and UNIX Case Study

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Protection & Security	2
1.1	Access Matrix Model	2
2	Introduction to Distributed Operating Systems	2
3	Case Study: The UNIX Operating System	2
3.1	System Architecture	2
3.2	The UNIX File System (Inode Structure)	3
3.3	UNIX Process Management & Scheduling	3

1. Protection & Security

Modern computer systems must protect data from unauthorized access, modification, and destruction.

- **Protection:** A mechanism for controlling the access of programs, processes, or users to the resources defined by a computer system.
- **Security:** A measure of confidence that the integrity of a system and its data will be preserved against external threats (viruses, hackers, physical damage).

1.1 Access Matrix Model

The **Access Matrix** is a formal protection model. It views the system as a set of **domains** (active entities, e.g., users, processes) and **objects** (passive hardware/software components, e.g., files, printers).

- **Grid Entry:** $\text{Matrix}[D, O]$ defines the access rights (read, write, execute) that process in domain D has over object O .
- **Implementations:**
 - **Access Control List (ACL):** Associates with each object a list of domains and their permitted rights (columns of the matrix). Used in Unix/Windows filesystems.
 - **Capability List (C-List):** Associates with each domain a list of objects and rights (rows of the matrix).

2. Introduction to Distributed Operating Systems

A **Distributed Operating System** manages a collection of independent, networked, and physically separate computational nodes, presenting them to the user as a single, unified computer system.

- **Network OS vs. Distributed OS:** In a Network OS, users are aware of the existence of multiple computers (e.g., must use `ssh` or remote mounting). A Distributed OS provides **location transparency** — the user cannot tell where a program is running or where a file is stored.

3. Case Study: The UNIX Operating System

UNIX is a highly modular, multitasking operating system originally developed at Bell Labs.

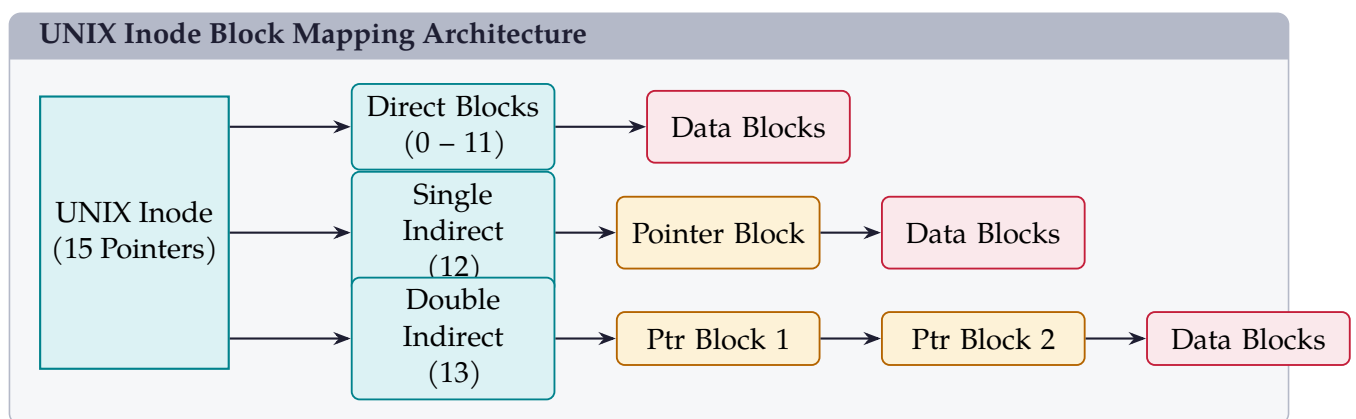
3.1 System Architecture

UNIX is divided into three main layers: 1. **User Space:** Applications, library calls, and the command interpreter (Shell). 2. **Kernel Space:** Manages process execution, memory, file systems, interrupts, and I/O. 3. **Hardware:** The underlying CPU, memory, and physical controllers.

3.2 The UNIX File System (Inode Structure)

In UNIX, every file is represented by a data structure called an **Inode (Index Node)**. The inode contains file metadata (owner, permissions, size, time stamps) and a table of **block addresses** pointing to the physical data on disk.

- **Inode Address Pointers Layout:** Typically contains 15 pointers:
 - **Pointers 0–11: Direct Blocks** (point directly to disk data blocks).
 - **Pointer 12: Single Indirect Block** (points to a block containing pointers to data blocks).
 - **Pointer 13: Double Indirect Block** (points to a block containing pointers to single indirect blocks).
 - **Pointer 14: Triple Indirect Block** (points to a block containing pointers to double indirect blocks).



3.3 UNIX Process Management & Scheduling

- **Process Creation:** Handled exclusively via `fork()` followed by `exec()`.
- **UNIX Process States:** Running in user mode, Running in kernel mode, Ready to run in memory, Asleep in memory (waiting for I/O), Ready to run swapped, Asleep swapped, Zombie (terminated but parent has not read exit status).
- **UNIX CPU Scheduling:** Uses a multilevel feedback queue scheduling algorithm. Priority is recalculated every second based on CPU usage:

$$\text{Priority} = \text{Base} + \frac{\text{Recent CPU Usage}}{2} + \text{Nice}$$

where `Nice` is a user-controllable process parameter (higher `Nice` value lowers priority).

Quick Revision — Module 9

Protection, Security and UNIX Case Study | OE-EC604B

UNIX Inode Max File Size Calculations

Let block size = 1 KB. Disk address pointer = 4 Bytes.

- **Max Direct Blocks Capacity:** $12 \times 1 \text{ KB} = 12 \text{ KB}$.
- **Max Single Indirect Capacity:** $(1 \text{ KB}/4 \text{ B}) \times 1 \text{ KB} = 256 \times 1 \text{ KB} = 256 \text{ KB}$.
- **Max Double Indirect Capacity:** $256 \times 256 \times 1 \text{ KB} = 64 \text{ MB}$.
- **Max Triple Indirect Capacity:** $256 \times 256 \times 256 \times 1 \text{ KB} = 16 \text{ GB}$.
- **Total Max File Size Limit:** $12 \text{ KB} + 256 \text{ KB} + 64 \text{ MB} + 16 \text{ GB} \approx 16.06 \text{ GB}$.

One-Line Definitions

- **Access Matrix:** Grid model representing domains (rows), objects (columns), and access rights (cells).
- **ACL:** Access Control List; column-wise access matrix stored on objects indicating permitted domains.
- **Capability List:** Row-wise access matrix stored inside process domains detailing object rights.
- **Distributed OS:** OS presenting multiple physical CPU nodes as a single logical computer transparently.
- **Location Transparency:** File or computation access method independent of physical node location.
- **Superblock:** UNIX filesystem metadata block containing system size, free block count, and inode limits.
- **UNIX Inode:** Index block detailing metadata and 15 storage pointers for a UNIX file.
- **Zombie Process:** Process that has terminated but still occupies an entry in the process table.

Frequently Asked / Viva Questions

Q1. Differentiate between Protection and Security in operating systems.

Protection is the internal OS mechanism to control process/user access to defined system resources. Security is the external defense mechanism protecting system integrity and data against threats like malware, hackers, and environment disasters.

Q2. Compare Access Control Lists (ACLs) and Capability Lists (C-Lists).

ACLs are column-oriented implementations of the Access Matrix; they associate permitted domains and rights directly with each object (e.g., file permissions).

Frequently Asked / Viva Questions (contd.)

C-Lists are row-oriented; they attach object rights to each domain/user process directly.

Q3. What is a Distributed Operating System? What are its primary advantages?

A Distributed OS coordinates multiple autonomous CPUs connected by communication networks, presenting them as a single logical machine. Advantages include resource sharing, computational speedup, reliability (fault tolerance), and scalability.

Q4. Explain the UNIX Inode pointer layout and calculate max file size for a 2 KB block size with 4-byte pointers.

Layout has 12 direct, 1 single indirect, 1 double indirect, and 1 triple indirect pointers. **Pointers per block** = $2\text{ KB} / 4\text{ B} = 512$.

Direct size = $12 \times 2\text{ KB} = 24\text{ KB}$.

Single indirect size = $512 \times 2\text{ KB} = 1024\text{ KB} = 1\text{ MB}$.

Double indirect size = $512^2 \times 2\text{ KB} = 512\text{ MB}$.

Triple indirect size = $512^3 \times 2\text{ KB} = 256\text{ GB}$.

Max File Size = $24\text{ KB} + 1\text{ MB} + 512\text{ MB} + 256\text{ GB} \approx 256.5\text{ GB}$.

Q5. What is the function of the Superblock in the UNIX file system?

The Superblock is located at a fixed offset near the start of the disk partition. It contains critical metadata about the file system, including total blocks, free block count, free inode list, size of inode list, and status flag.

Q6. Describe the UNIX process scheduling formula and explain the role of the nice value.

$\text{Priority} = \text{Base} + (\text{Recent CPU Usage} / 2) + \text{Nice}$. The OS decays priority based on CPU usage to prevent CPU starvation. The `Nice` value is an administrator/user offset (ranging from -20 to $+19$). A higher nice value increases the priority value, which actually lowers scheduling priority.

Q7. What is a zombie process in UNIX? How is it cleared?

A zombie process is a process that has finished execution but still has an entry in the system process table to allow its parent to read its exit code (via `wait()`). It is cleared when the parent executes a `wait()` system call. If the parent terminates, the zombie is inherited by the `init` process, which automatically calls `wait()`.

Q8. Differentiate between Loosely Coupled and Tightly Coupled distributed systems.

Loosely coupled systems have independent nodes with private memory connected via slow communication networks (e.g., LAN clusters). Tightly coupled systems share a common physical memory and bus architecture with fast inter-processor communication.

Q9. What is the Access Matrix model? How is it represented?

Frequently Asked / Viva Questions (contd.)

It is a security model where rows represent domains, columns represent objects, and cell entries represent the set of access rights (read, write, execute) allowed for a domain over an object.

Q10. Explain the role of the shell in the UNIX operating system.

The shell is a user-space application that acts as a command line interpreter, reading user commands, expanding wildcards, setting up I/O redirection/piping, and spawning processes using `fork/exec` to execute those commands.

Security Implementation Comparison

Feature	Access Control List (ACL)	Capability List (C-List)
Association	Attached to the Object (e.g. File)	Attached to the Subject (e.g. Domain)
Revocation	Easy (just delete domain from file list)	Hard (must search all domain lists)
Access check	Slow (must search file ACL at access)	Fast (domain presents token directly)
Common use	Standard OS filesystems	Object-oriented/microkernel systems